Project – "dynamic" traffic lights

Student – Beni Cherniavsky [1]
Project Supervisor – Ari Todtfeld [2]
Control, Robotics and Learning Systems Lab, [3]
Electrical Engineering, Technion, Israel

November 2002

[1]cben@tx.technion.ac.il
[2]todtfeld@tx.technion.ac.il
[3]http://www.ee.technion.ac.il/control/

Abstract

This project analyzed systems of traffic ligths, roads and cars, in search of the best startegy for controlling the traffic lights. The problem was analyzed and a simplified model was selected. Simulation software was developed for this model and run with many control strategies (due to time constraints only simple strategies were tried). This report documents what was done and analyzes the results.

Acknowledgements

I wish to thank my advisor, Ari Todtfeld, for guiding me; the Control, Robotics and Learning Systems Lab for providing this project and being ready to provide resources for it (even if I mostly prefered to work at home); and my brother Eli for tolerating that I grabbed the computer for long times `:-)`...

Also I want to thank Richard Stallman, Linus Torvalds, Guido van Rossum, Donald Knuth, Lesley Lamport and all other people who created GNU/Linux, Emacs, Python, LaTeX and all the great free software that I used...

Contents

1 Problem formulation **5**
- 1.1 The (too) big problem 5
- 1.2 What do we want? 6
- 1.3 Down to earth (the toy problem) 7

2 Project progress, issues and solutions **9**
- 2.1 Theoretical analysis 9
- 2.2 Software 10
  - 2.2.1 Implementation language 10
  - 2.2.2 Architecture 11
  - 2.2.3 Highlights 12

3 Simulation Results and Conclusions **16**
- 3.1 First Part – Probing the Ground 16
  - 3.1.1 Results 20
- 3.2 Second Part – the Heavy Car Crunching 22
  - 3.2.1 Results 24
- 3.3 Bottom lines and future directions 32

List of Tables

3.1 Reaction test numerical results, averaged over 10×10 & 15×15 cities. 23
3.2 Comparison of junctions with 3 and 4 roads, averaged over plans, selected parameters: *hold2*, *fix*, *lru* 25
3.3 Comparison of the 3 plans, selected parameters: *hold2*, *fix*, *lru*, *3* roads 26
3.4 Comparison of the 3 plans, selected parameters: *hold2*, *fix*, *lru*, *4* roads 27
3.5 Comparison of switching speeds, fixed and flexible, selected parameters: *0.20* load, *lru*, *4* roads, averaged over plans. Note that *flex* with *blind* makes no sense (it would be stuck on one queue indefinitely). 29
3.6 Comparison of control policies for different loads, selected parameters: *fix*, *lru*, *4* roads, averaged over plans. 31

List of Figures

3.1 Reaction test – car number: load=0.10, free, rand 17
3.2 Reaction test – average delay: load=0.10, hold=4, rand 18
3.3 Reaction test – total deviation: load=0.10, hold=4, rand . . . 19

Chapter 1

Problem formulation

In one sentence: given junctions, roads and cars find the best strategy for controlling the junctions. Definition of "best" will come later. Since the project was open-ended, it was needed to decide on the level of complexity with which to work. The problem eventually analyzed was, unsurprisingly, very much of a toy problem.

1.1 The (too) big problem

Let's first look at the most general (more or less) formulation of the problem to see what was omitted in the toy problem. It should be able to represent almost all aspects of traffic light control in real life, at the cost of being impossible to analyze `:-)`.

Our world ("city") consists of three main object types:

- Cars that move around. They can probably enter / leave the city at certain places. At junctions they decide what turns they should take.[1]
- Unidirectional roads that delay cars driving through them. The delay can depend on factors like amount of cars, random effects, etc.
- Junctions – our question is *How best to control them?*

[1] We assume that the choice of turns is independent from the delays on the road. Strictly speaking this is not correct — drivers try to stay away from jams they see ahead or hear about in traffic reports. However dropping this assumption would make comparison between control strategies problematic. For example if a person altered the order of his appointment to avoid a jam and thus lowered the total delay, it's a definite success of the reporting system but is it a success or a failure of the junction control system?

A given junction has M input roads and N output roads. There are $R \stackrel{\triangle}{=} MN$ routes for a car to pass through a junction (maybe less, for example certain left turns can be forbidden, etc.). A car arriving from input direction m and decides it wants to go to direction n are appended to queue $Q_{m,n}$ of the junction (not necessarily distinct – different ns can share the same queue). At any moment the junction is serving zero or more queues. "Serving a queue" means that from time to time (typically at a fixed rate for each queue, maybe after transition effects), one car (or more, think multiple lanes), is popped from the queue (assuming it's not empty) and forwarded to the target road(s) the car had chosen.

The choice of queue(s) to serve at a given moment is the subject of our analysis. Obviously it must be done under some constrains (otherwise we have an ideal interchange where all queues can be served at once and no project to do ;-)):

- Avoid collisions. For every two routes $r \stackrel{\triangle}{=} m \to n$, $r' \stackrel{\triangle}{=} m' \to n'$ to pass the junction, let $C_{r,r'}$ be 1 if the these car paths collide, 0 if they don't. Given this matrix we can find all *maximum sets of compatible routes* (a set S is such if $\forall\ r, r' \in S,\ C_{r,r'} = 0$ but $\forall\ \tilde{r} \in R - S\ \forall r in S,\ C_{\tilde{r},r} = 1$).

  Once computed (for given C), these sets redefine our task as choosing, at every point in time, which if these sets of queues to serve.

- Switching speed. Instantaneous switching from serving one set of queues to another would cause collisions just as well. So every switch is penalized with some dead time when nobody is served. Let's call this dead time *"yellow light"* even though it's not only it. Similarly, acceleration from stopped state further reduces the throughput for some time after a starts to be served. Actually, these effects happen per queue independent of other queues simultaneously served (which makes the above sets-of-queues switching model more complex). The net effect: the lower the switching rate the higher the total car throughput.

- Pedestrians. Every input and output path must be closed at least occasionally to allow for green lights for pedestrians too. Since junction designers don't seem to care much for them, this is the last mention of them in this project :-).

1.2 What do we want?

What are our criteria for measuring how "good" a given control policy is?

1. Most importantly, we want minimum total *delay* for a car along his way. If road delays are constant this boils down to the delay in the junction queues.

2. We also want *stability* (predictable delays are better than strongly fluctuating ones) and *fairness* (different cars should get similar treatment if possible).

Since different cars travel in different ways, it makes more sense to measure the average delay for each queue instead of for each car. Given these delays, we can try to define some effective delay of the city as some weighted combination of these.

Stability and fairness can be represented by the variance of the delay distribution. Here it isn't really valid to use the per-queue measurement approach if there is correlation between delays at different junctions. However comparing cars traveling in different ways would inflate the variance without reason. The proper definition is to measure variances between all cars the traveled the same routes (or at least shared route segments of some length). Since this would require tremendous simulation times to collect enough data and since in the end no control strategies deliberately creating correlation were used, the per-queue approach was used. (Some care has to be taken then when talking of the total variance over the city).

One more point to discuss: what input should be available to the control system? One can divide the research in this field into three somewhat separate parts:

- Given perfect information, develop the optimal control strategy.
- Check how sensitive the strategy is to imperfect information.
- Devise cheapest sensors giving maximum useful information.

This project belongs to the first part, so let's have full information `:-)`.

1.3 Down to earth (the toy problem)

The above model is way to complex to be useful. Let's throw away complicating details, until we get something manageable.

1. Cars decide where to turn statistically — fixed known[2] probabilities are assigned to the possible turns. This turning profile doesn't closely

[2] {60% straight, 20% right, 15% left, 5% reverse} were used in the simulations.

model real traffic (for example cars will readily enter dead ends or go in circles). The positive side is that an equal average car arrival rate at all junctions can easily be guaranteed. Thus we give up modeling of heterogeneous traffic load to be able to measure parameters as functions of the load.

2. Get rid of multiple lanes — all queues are served one car at a time with same constant speed.

3. Ignore acceleration issues. Use the "vertical queue" model — the cars stack on on top of the other at the end of the road instead of stretching back from the stop line. This is universally used in all traffic simulation literature I saw.

4. This is close to discrete time, so let's adopt it (much easier to simulate, control, measure...). Implications: serve one car per cycle, roads length are integral number of cycles, etc. One of the results is that at most one car per cycle can arrive from a every input road.

5. Roads have constant delays (no jams along the roads, only at junctions; no "platoon dispersion" which is actually important to consider but we don't).

6. No yellow lights. The original motivation for this was that to be able to control multiple junctions in some correlated manner, we will want to limit the switching rate to about the speed of passing an average road anyway. In the end only local control was implemented so this doesn't hold. Yellow lights were actually implemented in the simulator but not used due to time limits. Besides, the control strategies tried didn't take them into account anyway. Timing constraints were modeled by directly placing lower/fixed limits on the time each queue is served.

7. Serve only one queue at a time. Analyzing serving of sets of queues simultaneously turned out to be too complex. We do allow cars from one queue to serve many input directions (i.e. only one input road gets green light at the same time and cars can turn to all output directions). Note that this is a big departure from reality! It is suboptimal for typical junctions where most cars travel straight. *It limits the load* to be below $1/\max\{\text{inputs}_J \mid J \in \mathit{junctions}\}$ because otherwise that junction will accumulate cars faster than it can dispose of them and will become unstable. In our simulations this means that the load must be below 0.25.

Chapter 2

Project progress, issues and solutions

2.1 Theoretical analysis

First I tried to analyze a single junction. Consider the question of determining the best green time proportions for a *blind* (with no sensors, doesn't even know which queues are empty) junction with known arrival rates. Assume the junction can serve a different queue every cycle (no switching rate limits).

It knows only the arrival rates[1] $\{\lambda_m\}_{m=1}^M$ to the queues. Assuming it randomly choses the queue to serve each cycle with probabilities $\{\mu_m\}_{m=1}^M$, how to chose those probabilities (satisfying $\sum_{m=1}^M \lambda_m = 1$) to get minimal average delay?

This is a simple M/M/1 queues and optimization problem, resulting in:

$$\mu_m = \lambda_n + \sqrt{\frac{\lambda_m(1-\lambda_m)}{\sum_{m=1}^M \lambda_m(1-\lambda_m)}}$$

In other words, divide the extra serving rate $1 - \sum \lambda_m$ proportionally to the factors $\lambda_m(1-\lambda_m)$. This was later simulated by comparing junctions with optimal (according to the above formula) and suboptimal μ_n probabilities and optimal junction indeed gave lower delay.

Then I tried to analyze a junction that can serve several queues at once. This was described before and as I said I gave it up. Solving this for one junction (still blind) seemed possible but very hard. I realized that if I ever want to reach multiple junctions, I must simplify the single junction.

[1] note that a car can arrive together with departing, so the effective rates are $\lambda(1-\mu)$ and $\mu(1-\lambda)$.

I also considered the other details of the model (cycle length limits, yellow light, etc.) and more or less settled for the model discussed above at this stage.

Then I tried to consider non-blind junctions. The problem is that the probability to serve one queue depends (at least) on whether the other queues are empty. So I can't analyze queues independently. Just the first 4 states $((0,0),(0,1),(1,0),(1,1))$ of the state diagram for two queues resulted in a frightening diagram. It's probably solvable with some smart trick but becomes increasingly complex.

On the other hand I made a quick calculation of the probability that the above optimal blind junction decides to serve an empty queue (thus wasting throughput). To my horror it turned out to exceed 50% at some realistic parameters! So unless we only deal with heavy load (when queues seldom become empty), a blind junction is a terrible waste.

The conclusion from all this was that the time came to leave theory aside and write a computer program for simulation.

I also briefly looked at predicting which cars have good chances of being stuck at next junction and giving them lower priority (since serving them later probably won't increase their total delay). I didn't find an obviously good algorithm and dropped the issue deciding that I better implement at least something without prediction.

2.2 Software

2.2.1 Implementation language

I've chosen Python that I recently learned and came to like very much. It seemed well suited to the task at hand. From its `README`:

> **What is Python anyway?**
>
> Python is an interpreted object-oriented programming language suitable (among other uses) for distributed application development, scripting, numeric computing and system testing. Python is often compared to Tcl, Perl, Java, JavaScript, Visual Basic or Scheme. To find out more about what Python can do for you, point your browser to http://www.python.org/.

Was it worth it? Sure! I feel that the high abstraction level and quick development cycle Python provided has paid off many times. This project required many small tasks having to do with organizing the simulation runs and such a scripting language is perfect for them.

The performance became a bottleneck at the simulation running period but still I think it was worth the development ease. Besides the task is mostly symbolic computations and languages like C aren't that fast at them anyway.

2.2.2 Architecture

The problem at hand nicely lends itself to an object-oriented architecture (at which Python is very good, in my opinion; among others multiple inheritance was heavily used).

A good way to get a bird's eye view of the architecture is to describe the function of the modules composing the program:

utils Random functions (pun intended, one of them deals with random numbers).

textgrid Simple text formatting facility for printing grid pictures.

simtime Registry objects for city elements having .tick() methods that call them every cycle (in partially defined order).

connect Facilities for defining and conveniently connecting elements that have one or more named inputs and/or outputs.

basic Basic elements with one input and/or output: Car, Sources that produce cars, Drains that consume them, Road, Queues and simplest Servers.

multi Complex elements with many inputs and/or outputs: support base classes, RandomRouters and SwitchedServer.

city Almost the top module, imports the rest. Implement City objects that contain elements and GridPlan objects that represent subset-of-grid city geometries and can implement them by creating cities.

run Top file running the whole thing. Contains all the parameter choosing, running, logging and graphing logic.

mktable A stand-alone script that parses log files, treats them as multidimensional information and prepares beautiful LaTeXtables with different views of it (with value selection and averaging abilities).

2.2.3 Highlights

Here are some highlights of problems and their solutions in the simulator.

- Cars are passive object carried around by elements of the city. Their only methods are for statistics collecting.
- Proper ordering of actions was achieved by:
  - Procedural means in the simplest cases (for example injecting a car into some element is done by calling its .append(car) method which can react to this immediately).
  - The long-term time line is managed by a SimTime object (every city has one). Most elements have a .tick() method. The time manager calls this method of all objects that were registered with it every cycle. To make interactions during the same cycle well-defined, the objects are divided into several "phases" by their class and called with certain order between the phases (compare with "delta" times in VHDL simulation).
- Connecting elements is based on a Group interface (implemented by all elements) of an object which has named[2] input and/or output ports. Connecting resolves input/output names between two objects (like linking programs) and returns a virtual Group combining the unresolved ports of both objects. Virtual Groups are also used to implement renaming ports and "overlaying" objects without linking. These features proved very useful in the creation of complex cities.
- Junctions were implemented as consisting of two parts: a SwitchedServer with multiple inputs and one output connected to a RandomRouter with one input and multiple outputs. The serves serves at most one queue at a time. The router looks at the original direction recorded in the car by the server to choose probabilities for the possible outputs. There are also many slave objects to represent the queues and outputs.

  In the server, unlike most other objects in the program, I wasn't satisfied with sub-classing the base class to define the possible control strategies (even with multiple inheritance) because it didn't provide enough flexibility. Instead, a plug-ins-based architecture was employed. There is one SwitchedServer class that is parametrized with a list of filters. They are applied in order, starting with the full list of input queues, until one or zero candidates remain.

[2] A default name is used for single input/output cases

This way, the control strategy specification was split into many components, mostly orthogonal:

rand_filter Break ties by choosing a random queue.

lru_filter Break ties by choosing the least-recently-used queue. If previous filters (if any) result in stable queues set, this results in a round-robin behavior.

nonempty_filter Don't serve empty queues.

hold_filter Allow deciding on a queue only once in k cycles.

attempt_hold_filter Serve same queue that was served last time, unless it was dropped by previous filters (i.e. switch queue only when forced). In combination with hold_filter, allows to prolong serving of one queue to more than k cycles.

longest_queue_filter Serve the queue with most cars.

last_emptying_filter Serve queues predicted to empty last (if served continuously), based on input load measurements done by each queue. Queues measuring load by sliding window and exponential averaging were implemented[3]. For estimated arrival rate λ and serving rate 1, a queue of length L is expected to become empty after $L/(1-\lambda)$ cycles.

inorder_filter Serve the queue whose first car waited most. This was intended to be the most fair strategy theoretically possible by effectively treating all queues as one. Note that it is almost completely impossible to implement in real life. Among others, it doesn't have this property when switching rate is limited; however switching to a different queue each car is very impractical...

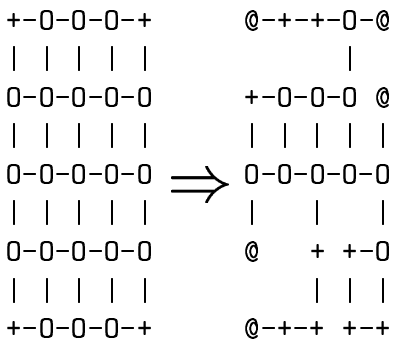
These filters can be combined in various creative ways; many but not all of them were exercised by the simulations.

Yellow light was implemented in the SwitchedServer class itself (it didn't fit the filters model conveniently enough) but as already said it wasn't used in simulations (it wasn't even tested).

- "Interesting" city geometries were generated without dealing with arbitrary graphs by starting with a full rectangular grid , dropping a certain portion (30% in the simulation) of the roads and treating junctions according to number of roads: 3 or 4 – remains a junction, 2 – join

[3] only exponential averaging was tried in simulations, with characteristic decay length of 4 cycles; it didn't make much difference anyway

the road into longer roads, 1 – a source/drain pair and for 0 nothing is created. All roads are bi-directional. Example:



(where `-` and `+` represent roads, `+` long roads join, `O` junctions and `@` source/drain pairs).

- Directions were represented with vectors – tuples of $(\delta x, \delta y)$. This proved much more useful then encoding them with, say $\{0, 1, 2, 3\}$, because they could be directly manipulated in many creative ways.

- To make comparisons between different strategies fair, the city geometry is generated in a plan object once and multiple cities can be instantiated based on the same plan.

- When running, progress and results are both printed to the screen and logged to a file too using a file-like `Log` class. Simple graphs are with a s function that writes `*.plt` and referred by them `*.dat` files and runs `gnuplot` to do the work.

- Since the run was very long (5 hours first part, above 2 days the second), I needed a way to stop and restart where I left. To this end:

  – City plans ascii-art dumps (in the above format) are written into `*.plan` files in the output directory; if the file already exists, it's loaded and the plan is set from it instead of being randomly generated.

  – When the log file is opened for appending, it's also scanned for headers of simulations already run. Before running every simulation, it's compared to the loaded list and skipped if already done.

These features made the annoyingly long runs quite tolerable...[4]

- Simulation parameters are defined with nested loops over different aspects of the parameters. Every aspect's possible values have short symbolic names which, combined into one string, are used to name output files and identify the parameter sets in graphs and logs. For example: *0.20-hold4_fix_rand-C*.
- "Visualization" of numbers in nice tables was implemented in a separate log→tables converter script which knows to nest tables to show more than 2-dimensional data.

You are referred to the source code for the full details. The source code contains pretty good documentation that should be enough for understanding and modifying the code. It does not contain enough documentation for interfacing to the code without reading it, simply because it provides no such interface `:-)`. There is no clear implementation/user division line. Levels of convenience were implemented, until I was able to run the thing conveniently (with the Python prompt as my interface). If you just want to reproduce my results, its enough to say at the Python prompt:

```
>>> from run import *
>>> all_react()
>>> all_main()
```

(be prepared for a couple days of running!). However if you want alter simulation parameters, you need to modify the code — from superficial `run.py` modifications to writing new classes, depending on how much you want to change.

[4] Also, the just-in-time optimizer for Python, Psyco (`http://psyco.sf.net`) was used and seemed to give a speedup of somewhat below 2, lower than its average performance but still very welcome `:-)`

Chapter 3

Simulation Results and Conclusions

3.1 First Part – Probing the Ground

The main purpose of the first serious run was to find out good parameters for subsequent runs.

One issue that interested me was the validity of averaging data over some time that includes transition effects in the beginning, until the initially empty city stabilizes around a given number of cars. This was checked by simulating cities and recording the number of cars at every moment. It was later plotted (see Figure 3.1 on the following page for example) so that the reaction time to turning the sources on at $t = 0$ can be visually inspected. Similarly, at $t = 4000$ cycles, all sources were turned off and simulation continued until $t = 6000$; the reaction time to this too can be inspected.

The second thing checked was how long do I need to run to get sensible results. This was visualized by plotting the average and deviation over the whole city, accumulated from $t = 0$ as function of t (see Figures 3.2 to 3.3 on pages 18–19 for example).

The third thing checked was how big a city is needed. Every set of parameters was tried on 3 cities of sizes 5×5, 10×10 and 15×15. Each of the above plots was combined on one sheet for the three cities.

Last but not least, the volume of the run wasn't small. It ran for about 5 hours[1]. All of the above was repeated for 3 load values (0.05, 0.10, 0.20[2]) and a fair number of control policies. The collected statistics are instructive on their own.

[1] All run times are reported from my 933MHz Pentium III

[2] Recall that 0.25 would saturate the system

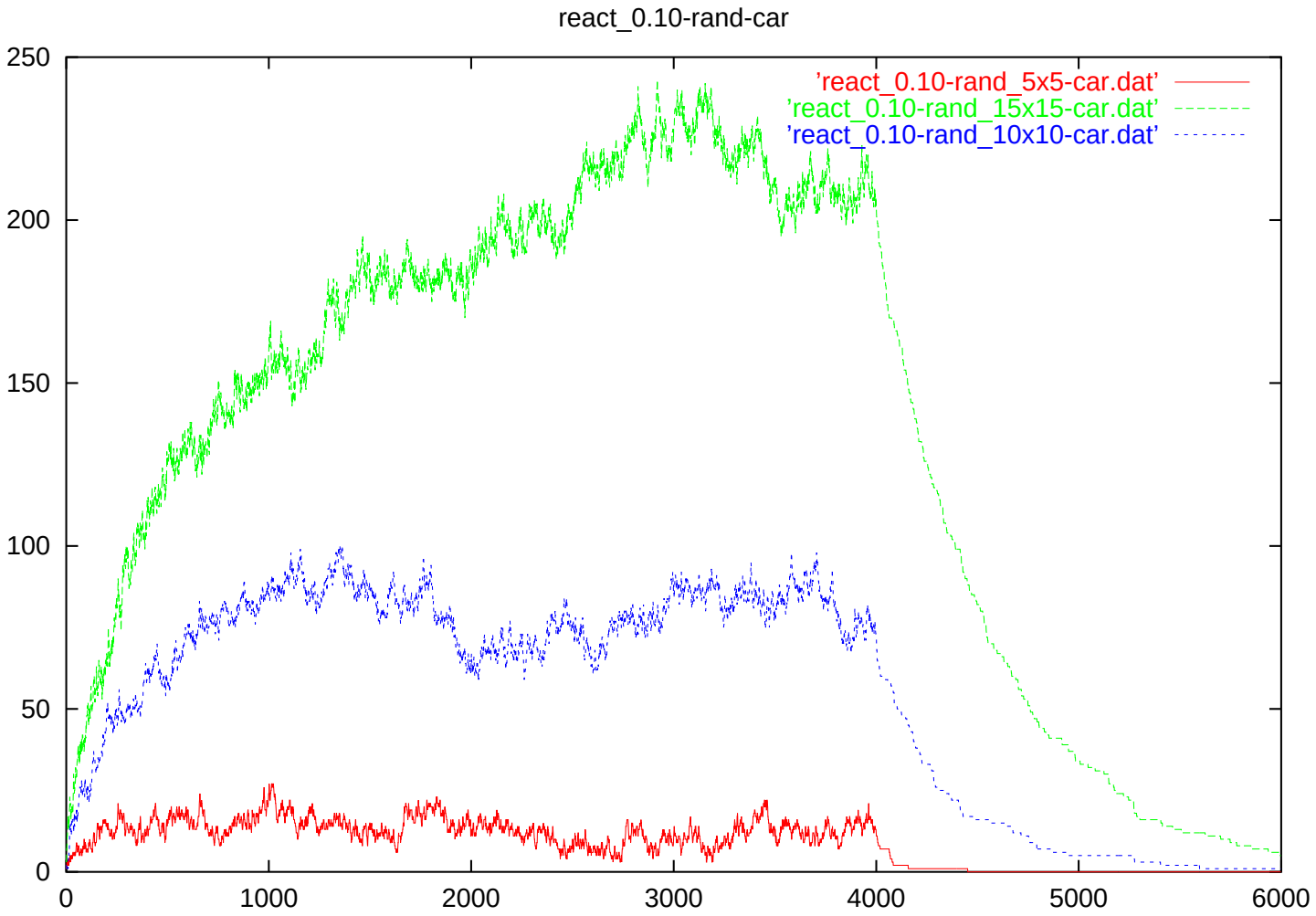


Figure 3.1: Reaction test – car number: load=0.10, free, rand

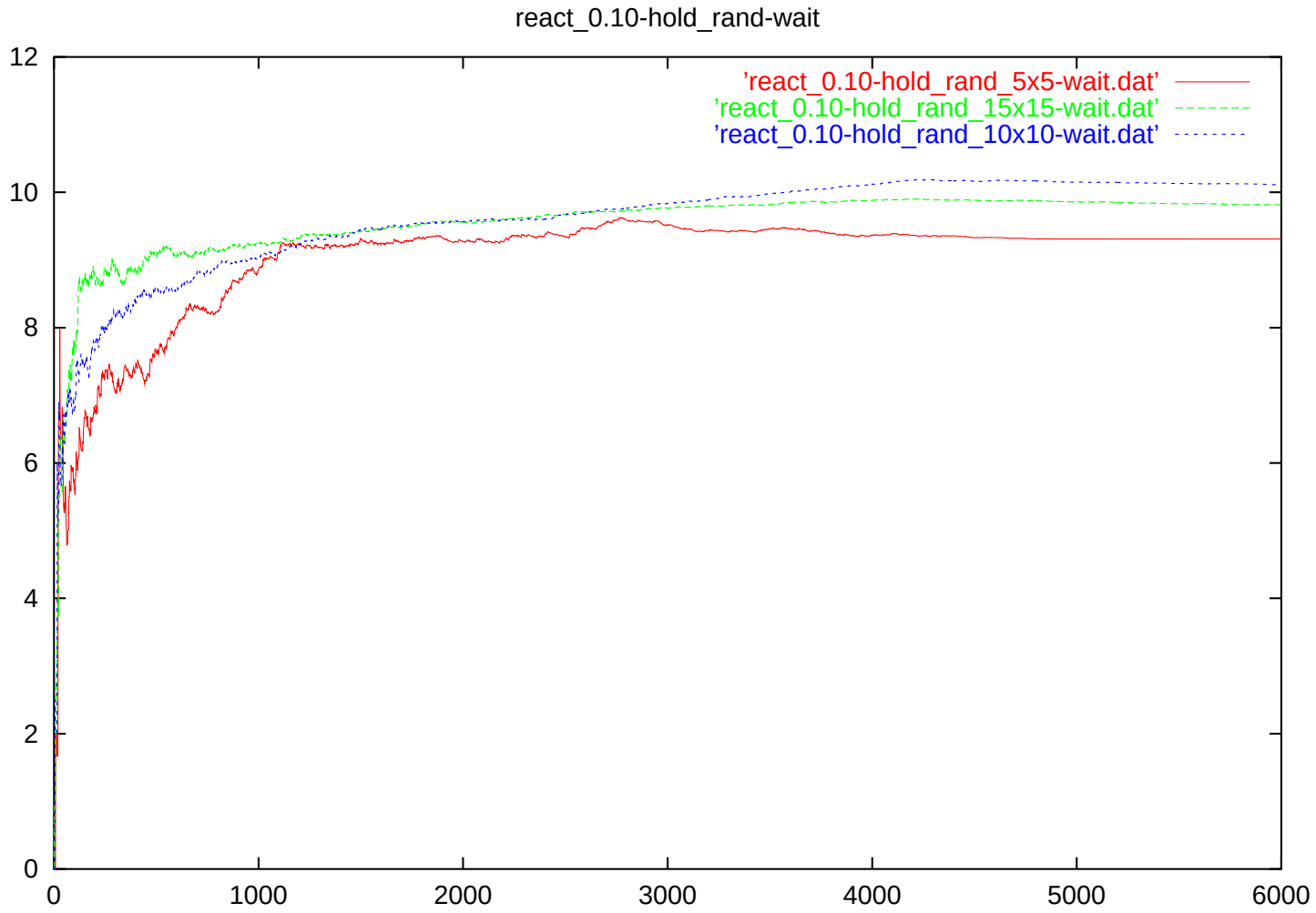


Figure 3.2: Reaction test – average delay: load=0.10, hold=4, rand

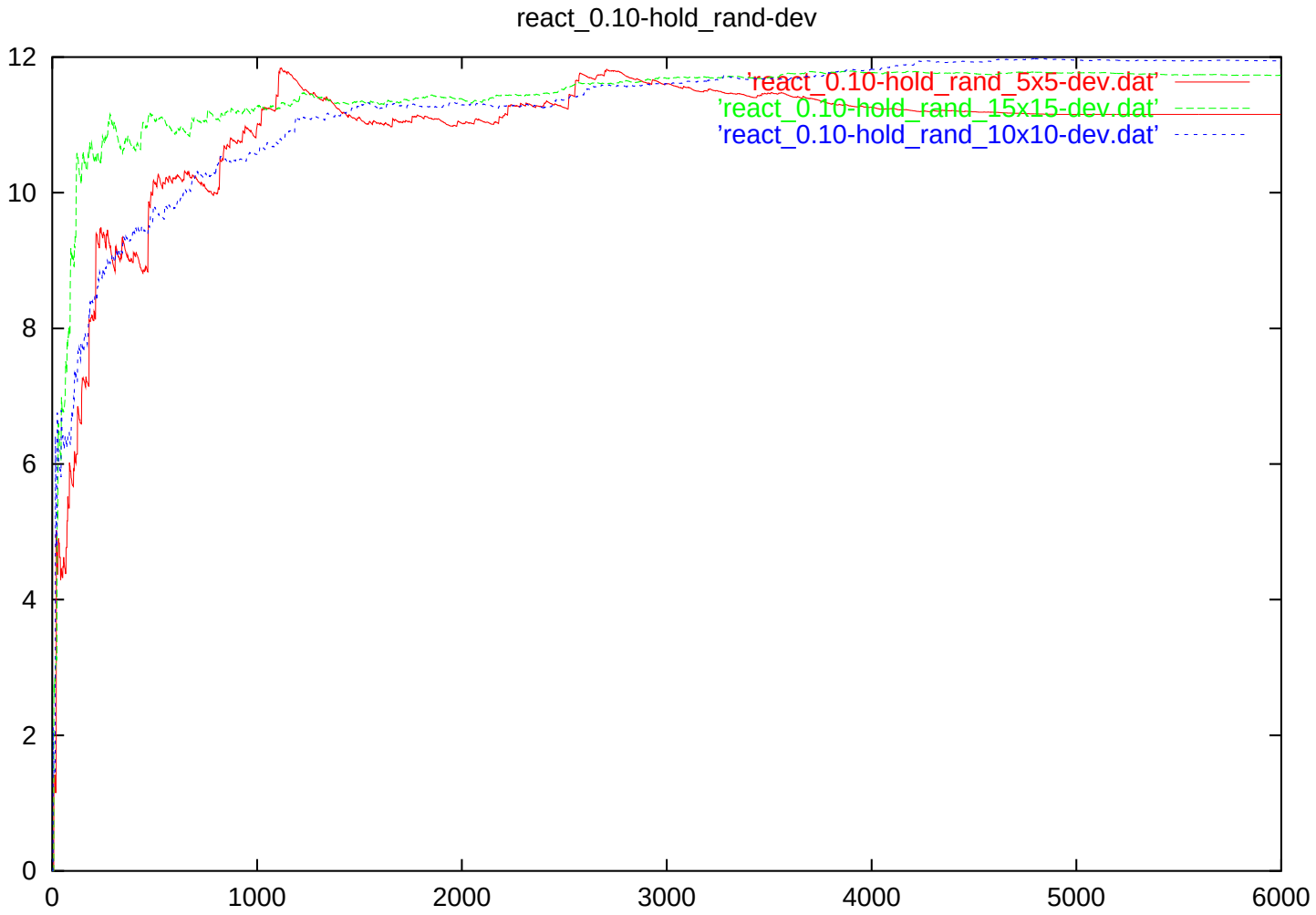


Figure 3.3: Reaction test – total deviation: load=0.10, hold=4, rand

3.1.1 Results

First, the qualitative conclusions:

⇒ The number of cars typically stabilizes after no more than 1000 cycles. This doesn't seem to contribute much to the final values – the curve becomes flat. Conclusion: *it seems OK to include the startup period in measurement.*

⇒ After 5000 cycles the resulting delay and deviation become as stable as they can get. Since the deviation doesn't go down, we know this is the "inherent" deviation of the system and result from low amount of measurement. *5000 cycles will be used for second part of measurements.*

⇒ The 10×10 and 15×15 cities differ little one from the other but differ seriously from 5×5. The problem with 5×5 is that it is too small for reliable statistics. For example I saw frequently 5×5 cities torn into two halves, etc. *To be on the safe side, 15×15 will be used for the second part.*

The numerical results are summed up in Table 3.1 on page 23, based on short identifiers describing different aspects of the run parameters. There is the load of course; there is also the plan name (written last). The rest describes the control policy: `hold` means that switching was limited to once in 4 cycles (in this part), `lru`/`rand` are the blind policies, `longest`, `hist`, `inorder` are the queue-inspecting policies described before. `avg` and `dev` are obviously the average and the deviation.

Analysis of the numerical results:

⇒ We see, as can be easily expected, that the three smart policies (which all don't serve empty queues) have almost equal delay. This makes sense – the average delay depends only on the portion of throughput wasted by serving empty queues. The differences between them are very small and don't seem systematic so they should be attributed to random variations.

⇒ We see that LRU and random serving are both much worse but while LRU is still in some acceptable bounds, the random policy gives completely horrible results. It can be intuitively expected that LRU which cover queues more systematically, will leave less cars to wait for long; nevertheless the fact that *blind LRU's delay is closer to that of controls with empty queue sensors* is very surprising to me. I'm not sure how to explain that big difference.

⇒ We see that the blind policies have bigger deviations than the ones with sensors. This is reasonable, especially since the values are all on the positive side of the zero (so typical probability distributions have give higher deviations with high higher means).

⇒ We see that the in-order serving policy gives smaller deviation than all others. This is consistent with the predictions – it was designed precisely for that – to result in lowest theoretically possible deviation.

⇒ The longest queue policy and the arrival rate measuring policy give practically equal results. Actually they are very similar: the later just divides the queue length by $1 - \lambda$. Since in our simulations the load is approximately equal on all inputs, we see no big difference. Conclusion: *There is no strong autocorrelation in the arrival rates in this system* so the measurements of the past give little information about the future.

⇒ The results with switching rate limited to once per 4 cycles are quite similar, except for the overall higher delay and waiting time. This is not very surprising but not obvious either – the "smart" policies can't be expected any longer to give equal delays. Indeed they don't but the results are not dramatically conclusive.

Roth policies based on the queue length give very close results. This is a bit unexpected since the with limited switching rate, the arrival rate should have a certain autocorrelation. The probable explanation is that the measurement scheme which has a reaction time of the same order as the autocorrelation time, (add the road delay to it) is reacting too late. *Communication between junctions should be able to provide an advantage here.*

There are noticeable differences between the in-order policy and the queue-length based policies. They are surprising: for very low loads[3], serving the car waiting the most is *not only more fair but also seems to have a predictive quality* for the future beyond that given by the current queue length! On the other hand for higher loads, serving in order of arrival starts to be worse than looking at the queue length.

On a second thought it makes sense: when queues rarely exceed a length of 1 car, the arrival time is an extra information that is not available to the length-of-queue based strategy. However the length of the queue is much more solid promise for less serving of empty queues — when

[3]To get a feeling: with switching every 4 cycles and arrival rate of 0.05, less than one car arrives on average during each red light period!

there is a difference in length it's more important than arrival times of the first cars in each queue...

3.2 Second Part – the Heavy Car Crunching

With the experience from the first run described above, a more heavy and exhaustive simulation was run, with the following parameters:

- 15×15 cities.
- 3 different plans of this size (called `A`, `B` and `C`) were simulated. This is intended to determine what fluctuations derive from the random city geometry and what is defined by the other parameters.
- Each run is for 5000 cycles.
- Higher load values: [0.15, 0.2, 0.23]. The loads considered previously are a bit too low. We wanted to cover these loads too.
- More control policies. All combinations of the following[4] were tried:
  - *free*, *hold2*, *hold4*, *hold4* — Switching rate is unlimited or limited to once per 2, 4 or 8 cycles.
  - *flex* — Serve same queue as last time, if possible. In combination with *hold*k, also means that the switching rate limit will be flexible – if the queue is not empty after k cycles, continue serving it.

    fix — the opposite. Note that the same queue can still be re-selected for another k cycles – but only on $t \bmod k = 0$ boundaries.
  - *blind* — Proceed directly to LRU/random resolution below.

    nonempty — Only try to serve non-empty queues; no preference beyond that.

    longest — Choose the queue(s) with most cars.

    hist — Choose the queue(s) predicted to become empty last if served continuously.

    inorder — Choose the queue(s) whose cars arrived first.
  - *rand* — If the above selection left several equally good queues, resolve by choosing at random.

    lru — Resolve ties in LRU manner.

[4] except for *blind* combined with *flex* which would serve one queue indefinitely...

	0.05			*0.10*			*0.20*	
free		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	hist	1.079	0.597	*hist*	1.211	0.812	*hist*	2.069	1.559
	inorder	1.070	0.528	*inorder*	1.201	0.718	*inorder*	2.092	1.368
	longest	1.062	0.548	*longest*	1.224	0.827	*longest*	2.133	1.599
	lru	1.327	1.133	*lru*	1.712	1.372	*lru*	3.483	2.189
	rand	3.817	1.851	*rand*	4.525	2.069	*rand*	6.720	2.718
hold		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	hist	3.078	1.534	*hist*	3.675	1.767	*hist*	5.064	2.091
	inorder	2.367	1.378	*inorder*	3.233	1.603	*inorder*	5.269	1.957
	longest	3.037	1.508	*longest*	3.660	1.760	*longest*	4.972	2.069
	lru	4.189	1.851	*lru*	4.426	1.885	*lru*	5.831	2.361
	rand	8.965	3.327	*rand*	9.963	3.441	*rand*	13.480	3.997

Table 3.1: Reaction test numerical results, averaged over 10×10 & 15×15 cities.

These short names will be used extensively in the following discussions of the results.

- Separate statistics for junctions with 3 and 4 roads. The motivations is that given equal average load at all roads, junctions with 4 roads experience higher total load than ones with 3 roads. This can be responsible for part of the deviation. Collecting statistics separately will not suffer from this "false" deviation.

These parameters combine to amount for a very long run. There are 72 different policies; multiplied by 3 load values and 3 plans, we get 648 runs. The sum of the counts of cars that passed through each junction (i.e. each car is counted many times) is 207,140,557 (two hundred millions) :-). The total running time exceeded 48 hours (I don't have exact figures because thanks to the restart-ability facilities, I was able to interrupt for rebooting to a lesser OS several times).

3.2.1 Results

The data from this run is essentially 8-dimensional. Displaying all of it at once is impossible and incomprehensible. So different projections/cuts of it will be presented, with arbitrary choices (or averaging where sensible) imposed on parameters that give less insight to the issues at hand, in order to reduce the number of dimensions.

⇒ Table 3.2 on the following page compares between statistics for junctions with 3 and 4 roads. It clearly shows that the suspicions were true. *There is a large difference between the statistics over junctions having 3 and 4 roads*, because they have different load. The lesson to take from this is that for precise measurements, they should not be mixed. So, in most following checks, I chose to concentrate on junctions with 4 roads (we wanted higher loads after all).

⇒ Now let's check the dependence on a specific plan. Tables 3.3 to 3.4 on pages 26–27 compares the 3 city plans, for junctions with 3 and 4 roads respectively (in this case the results were interesting enough to include both of them). There are definite and quite systematic differences.

For 3 roads (all loads) or 4 with the lower load of 0.15, the ordering of delays A > B > C is observed without exception. Interestingly, in all cases but one B has the highest deviation. But now *there is a breaking point on the load×number-of-roads scale* at which the relation between the city plans changes.

	0.15		*0.20*		*0.23*	
blind		*avg*		*avg*		*avg*
	3	2.671	*3*	3.286	*3*	3.675
	4	5.006	*4*	7.621	*4*	10.756
		dev		*dev*		*dev*
	3	1.417	*3*	1.677	*3*	1.842
	4	2.043	*4*	2.743	*4*	3.535
hist		*avg*		*avg*		*avg*
	3	1.377	*3*	1.857	*3*	2.136
	4	2.188	*4*	3.697	*4*	5.339
		dev		*dev*		*dev*
	3	1.208	*3*	1.381	*3*	1.466
	4	1.536	*4*	1.968	*4*	2.498
inorder		*avg*		*avg*		*avg*
	3	1.434	*3*	1.942	*3*	1.972
	4	2.329	*4*	3.937	*4*	5.375
		dev		*dev*		*dev*
	3	1.140	*3*	1.294	*3*	1.400
	4	1.413	*4*	1.757	*4*	2.186
longest		*avg*		*avg*		*avg*
	3	1.392	*3*	1.853	*3*	2.060
	4	2.208	*4*	3.666	*4*	4.759
		dev		*dev*		*dev*
	3	1.212	*3*	1.369	*3*	1.453
	4	1.535	*4*	1.981	*4*	2.359
nonempty		*avg*		*avg*		*avg*
	3	1.470	*3*	2.058	*3*	2.068
	4	2.623	*4*	4.877	*4*	7.228
		dev		*dev*		*dev*
	3	1.238	*3*	1.457	*3*	1.577
	4	1.661	*4*	2.295	*4*	2.964

Table 3.2: Comparison of junctions with 3 and 4 roads, averaged over plans, selected parameters: `hold2`, `fix`, `lru`

	0.15			*0.20*			*0.23*	
blind		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	A	3.285	1.395	*A*	3.766	1.618	*A*	3.986	1.683
	B	2.457	1.473	*B*	3.209	1.787	*B*	3.881	2.108
	C	2.272	1.384	*C*	2.882	1.627	*C*	3.159	1.736
hist		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	A	1.960	1.184	*A*	2.405	1.332	*A*	2.607	1.414
	B	1.133	1.238	*B*	1.599	1.427	*B*	2.066	1.552
	C	1.039	1.203	*C*	1.567	1.383	*C*	1.735	1.433
inorder		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	A	2.029	1.130	*A*	2.428	1.258	*A*	1.769	1.342
	B	1.244	1.177	*B*	1.835	1.355	*B*	2.249	1.478
	C	1.028	1.113	*C*	1.564	1.268	*C*	1.899	1.380
longest		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	A	1.996	1.195	*A*	2.407	1.338	*A*	2.557	1.409
	B	1.151	1.243	*B*	1.653	1.404	*B*	1.885	1.504
	C	1.030	1.198	*C*	1.500	1.365	*C*	1.738	1.447
nonempty		*avg*	*dev*		*avg*	*dev*		*avg*	*dev*
	A	2.023	1.205	*A*	2.601	1.435	*A*	1.783	1.489
	B	1.237	1.279	*B*	1.913	1.518	*B*	2.348	1.647
	C	1.149	1.230	*C*	1.661	1.419	*C*	2.072	1.594

Table 3.3: Comparison of the 3 plans, selected parameters: `hold2`, `fix`, `lru`, `3` roads

	0.15		*0.20*		*0.23*	
blind		*avg*	*dev*	*avg*	*dev*	*avg*	*dev*
	A	5.314	1.917	6.521	2.327	7.916	2.720
	B	5.154	2.208	9.209	3.220	13.939	3.991
	C	4.550	2.005	7.134	2.683	10.412	3.893
hist		*avg*	*dev*	*avg*	*dev*	*avg*	*dev*
	A	2.511	1.433	3.883	1.850	4.871	2.284
	B	2.180	1.626	3.717	2.053	6.325	2.668
	C	1.872	1.549	3.492	2.002	4.820	2.542
inorder		*avg*	*dev*	*avg*	*dev*	*avg*	*dev*
	A	2.796	1.385	3.663	1.572	4.128	1.964
	B	2.387	1.505	4.551	1.926	7.189	2.552
	C	1.805	1.348	3.596	1.773	4.809	2.041
longest		*avg*	*dev*	*avg*	*dev*	*avg*	*dev*
	A	2.666	1.476	3.642	1.790	4.327	2.046
	B	2.153	1.613	4.048	2.167	5.119	2.478
	C	1.804	1.517	3.308	1.987	4.830	2.554
nonempty		*avg*	*dev*	*avg*	*dev*	*avg*	*dev*
	A	2.899	1.549	4.242	1.950	4.491	2.364
	B	2.515	1.713	6.114	2.685	10.342	3.511
	C	2.455	1.721	4.275	2.251	6.852	3.018

Table 3.4: Comparison of the 3 plans, selected parameters: `hold2`, `fix`, `lru`, `4` roads

For 4 roads with the higher loads 0.20 and 0.23, the trend changes. B has the highest delay with no exceptions; it also has the highest deviation except for one case. A and C are quite close except for the blind case which has really big delays and differences; there doesn't seem to be a definitive ordering between them.

The question is, what properties of a city plan make it have overall higher or lower average/deviation? I must admit I found no good answer to this. I counted the components — sources/drains, long roads, junctions with 3/4 roads and total junctions — of the city plans:

	0	1	2	3	4	3+4
A	3	16	73	96	37	133
B	3	30	78	76	38	114
C	9	21	68	89	38	127

None of these seems to explain directly the A > B > C ordering observed for lower loads. However one thing does stand out: B has the highest number of sources/drains and highest number of road continuations, at the expense of junctions. How can this explain B's deviation being the highest? The more sources/drains, the shorter is the average car's presence in the city — but this shouldn't matter directly because measurements are per queue. Another difference is that incoming car streams have less autocorrelation (our sources emit white noise).[5] Road lengths shouldn't have direct effect on the control policies described. So, without proof, I can *guess: cars randomly entering/leaving the roads lower the autocorrelation of the car streams in the system which has adverse effect on the deviations.*[6]

In the face of these differences between cities, to what extent can we trust other information extracted from the data? First, can we bound the differences due to the plan? No, not tightly enough. The differences in many cases are as high as one third! However, we are saved by the fact that, as can be seen from the tables, the relations between different parameters are very similar for all three city plans. So while *absolute results for given cities are not representative, relative results are probably reliable.* From now on I will average results between the 3 plans.

[5]Recall that these tables are for switching speed of once in 2 cycles, which imposes at least some autocorrelation on the streams of cars emitted by junctions.

[6]Prediction: "platoon dispersion" on real-life roads probably has similar effect.

	free		*hold2*		*hold4*		*hold8*	
fix		*avg*		*avg*		*avg*		*avg*
	blind	6.609	*blind*	7.621	*blind*	9.747	*blind*	14.655
	hist	2.041	*hist*	3.697	*hist*	5.935	*hist*	10.784
	inorder	2.021	*inorder*	3.937	*inorder*	7.256	*inorder*	12.385
	longest	2.061	*longest*	3.666	*longest*	6.009	*longest*	10.913
	nonempty	2.289	*nonempty*	4.877	*nonempty*	7.967	*nonempty*	12.833
flex		*avg*		*avg*		*avg*		*avg*
	blind	—	*blind*	—	*blind*	—	*blind*	—
	hist	2.151	*hist*	3.362	*hist*	5.086	*hist*	9.231
	inorder	2.313	*inorder*	3.530	*inorder*	6.221	*inorder*	11.165
	longest	2.011	*longest*	3.252	*longest*	4.997	*longest*	9.343
	nonempty	2.132	*nonempty*	3.614	*nonempty*	5.896	*nonempty*	10.646

Table 3.5: Comparison of switching speeds, fixed and flexible, selected parameters: `0.20` load, `lru`, `4` roads, averaged over plans. Note that `flex` with `blind` makes no sense (it would be stuck on one queue indefinitely).

⇒ Now, let's consider the effects of different switching speeds, both fixed and flexible. Table 3.5 on the page before shows them for one load (0.20), with LRU tie resolving. The first thing to see is that for most cases, flex gives lower times (by up to $\approx 20\%$). This makes sense. The average delay should only depend on the frequency of serving empty queues. For any non-blind policy with fixed switching rate k, this can only happen on cycles $2 \ldots k$ out of every k cycles. For a flexible switching rate, if the period of serving a queue is prolonged by l cycles, a non-empty queue can be served again only on cycles $2 \ldots k$ out of the $k + l$ cycles. So *flexibly prolonging green time when the queue still has cars lowers the average delay by lowering the opportunity of serving empty queues.* The same can be expected of flexibly shortening the serving time. Indeed, this is observed: the average delays drops with the reduction of the minimal green time. However, *with no minimal green time limit (`free`), trying to prolong the green time is not efficient* — `fix` gives lower delay than `flex`. I don't know why exactly this happens — for `free` empty queues are never served so the delay depends only on complex second-order timing interactions between junctions...[7]

Another effect is that `flex` results in higher deviations. This is expected — it serves the same queue longer, making other queues wait more.

⇒ Finally, table 3.6 on the following page compares the different control policies, for different loads and switching rates. The results are similar to the ones from the first (reaction speed) runs; I won't repeat all the previous conclusions here. The main difference is that higher loads were used, so the results are more fluctuating. In particular, even for `free` the results are noticeable (but not critical) differences between the non-blind policies. This can be attributed to second-order interactions.

We see that *with the increase of load, the differences between the policies don't seem to become smaller.* In particular, `blind` remains very ineffective. This was unexpected — I would think that when most queues are full, a blind junction "has no choice" but to be effective too `:-)`. Probably the loads are still not high enough — with the smart policies, cars wait up to 4 cycles on average, which means 1-2 cars per queue in steady state...

[7] By these I mean the correlations between car arrivals from different directions to the same junction; understanding them falls beyond the scope of this project.

	free		*hold2*		*hold4*		*hold8*	
0.15		*avg*		*avg*		*avg*		*avg*
	blind	4.039	*blind*	5.006	*blind*	7.535	*blind*	13.349
	hist	1.400	*hist*	2.188	*hist*	4.592	*hist*	8.910
	inorder	1.111	*inorder*	2.329	*inorder*	5.677	*inorder*	10.564
	longest	1.161	*longest*	2.208	*longest*	4.872	*longest*	8.870
	nonempty	1.082	*nonempty*	2.623	*nonempty*	5.551	*nonempty*	10.442
0.20		*avg*		*avg*		*avg*		*avg*
	blind	6.609	*blind*	7.621	*blind*	9.747	*blind*	14.655
	hist	2.041	*hist*	3.697	*hist*	5.935	*hist*	10.784
	inorder	2.021	*inorder*	3.937	*inorder*	7.256	*inorder*	12.385
	longest	2.061	*longest*	3.666	*longest*	6.009	*longest*	10.913
	nonempty	2.289	*nonempty*	4.877	*nonempty*	7.967	*nonempty*	12.833
0.23		*avg*		*avg*		*avg*		*avg*
	blind	10.072	*blind*	10.756	*blind*	12.974	*blind*	16.977
	hist	3.517	*hist*	5.339	*hist*	6.850	*hist*	12.306
	inorder	3.780	*inorder*	5.375	*inorder*	9.042	*inorder*	13.740
	longest	4.050	*longest*	4.759	*longest*	7.418	*longest*	12.562
	nonempty	3.611	*nonempty*	7.228	*nonempty*	10.780	*nonempty*	15.176

Table 3.6: Comparison of control policies for different loads, selected parameters: `fix`, `lru`, `4` roads, averaged over plans.

We see that *with stronger limits on the switching rate, all policies become more similar*. This is expected — none of them is can predict the future very well so they all are more or less blind with respect to it.

Again, the differences between `hist` and `longest` are very small and unsystematic; considering the fluctuations in the results, there is no statically meaningful difference between them. Under the conditions of these simulations, *measuring the incoming load of each queue doesn't prove useful* (or maybe the information wasn't put to good use).

Continuing the trend from the first runs, `inorder` lags behind them and its delay isn't different from `nonempty`. Serving in order of arrival doesn't improve the delay, except under the smallest load. However it does improve the deviation better than any other method. Here is a clear *delay/fairness trade-off*.

The rest is consistent with the conclusions from the first run. There are surely points I missed in the analysis and other interesting tables to produce but I have to stop somewhere. You are invited to mine the results further ;-).

3.3 Bottom lines and future directions

First, avoid serving empty queues as much as possible. If you can't switch the served queue every cycle (e.g. in real life), choosing the longest queue is a good strategy to minimize it. For very low loads, when several queues have equal length, breaking the tie by choosing the one whose front car arrived first should be better yet.

Second, flexibly controlling the green time is another way to reduce it. Serve a queue as long as it has cars, when it empties, switch to another. The lower the minimum green time limit, the better (unless you are penalized by yellow light time). A limit on maximum green time would be useful if you don't want to harm fairness.

Third, always prefer least-recently-used approaches to picking at random. This helps both for delay and fairness. There is some trade-off between delay and fairness. Serving in order of arrival is always most fair[8]. However it is not very practical since for `hold` it gives suboptimal delay.

There are many ways to continue this work — it's just the tip of the iceberg. One could perform more research with the current simulator. In particular, the conditions tried were still not representative of real-life traffic;

[8]For `free`, it is a special case of the PGPS weighted fair queuing algorithm, for equal ϕ_i weights.

higher loads, longer green times, longer roads and yellow lights would be interesting to explore. The role of the city's geometry should be explored more; the current Manhattan-like geometry is very arbitrary.

It would be interesting to implement logging/visualization facilities that would allow one to track the history of a single car (or a little group) and get a feeling of the system's behavior. This could help to gain intuition for second-order effects.

Smarter car routing would be interesting; a simple approach is to choose a route like now when a car is created and remove all cycles from it.

Each of the assumptions described at the first chapter can be considered for lifting. Serving multiple queues at once would be most valuable for getting closer to reality; unfortunately it will probably severely complicate all control policies.

The ultimate goal is probably devising control policies that involve communication between junctions.

Use your imagination to continue this list...